

Homework 1

Due on *myCourses* Wednesday September 30, 9:00pm.

General instructions.

- This is an individual assignment. You can discuss solutions with your classmates, but should only exchange information orally, or else if in writing through the discussion on *Ed* or in an assignment help sessions with a TA. All other forms of written exchange are prohibited.
 - If you did attend an assignment preview session and collaborated with someone (took notes away from your session), please write down the name and email of the person you worked with.
- Unless otherwise mentioned, the only sources you should need to answer these questions are your course notes, the textbook, and the links provided. Any other source used should be acknowledged with proper referencing style in your submitted solution.
- For each problem, you can solve manually, or write a program to help you. You can use a programming language of your choice. You can modify code from other sources if you provide adequate citation; this cannot be code from other students in the class.
- Submit a single pdf of your written responses through *myCourses*. You can scan-in hand-written pages. If necessary, learn how to combine many pdf files into one.
- In addition, submit any code developed to answer questions (e.g., Question 3) as a separate file through *myCourses*.

Question 1: Six-Puzzle [30]

Write code in a language of your choice to solve this problem. Hand in the source code with your submission, making clear that the file(s) is/are for Question 1.

Consider a variant of the eight-puzzle, which we saw in class, where we reduce the size of the puzzle to 3x2, in order to keep the number of states manageable. Now, consider the following initial and goal states:

Initial state:

1	4	2
5	3	

Goal state:

	1	2
5	4	3

- Show the solution path (i.e., the sequence of puzzle states from the initial to the goal state) found by each of the following algorithms, assuming transitions have unit cost. **You must ensure that puzzle states that have been explored are not added to the search queue.** Given multiple states to explore that are otherwise equivalent in priority, the algorithm should prefer the state that involves moving a lower-numbered piece.
 - Breadth first search

- ii. Uniform cost search
- iii. Depth first search
- iv. Iterative deepening

- b) Suppose now that transitions have differing costs. In particular, the cost of a transition is equal to the number of the piece that is moved (e.g., moving the “4” costs 4). If we employ the Manhattan distance heuristic for the original unit cost version of the eight-puzzle presented in class, would this heuristic still be an admissible heuristic for A* search in the new variant? Justify your answer.
- c) Design an admissible heuristic that dominates the heuristic from part b, under the same transition cost scheme as part b.

Question 2: Search algorithms [30]

(Adapted from Russell & Norvig)

- a) Describe a state space in which iterative deepening search performs much worse than depth-first search (for example, $O(n^2)$ vs $O(n)$).

Prove each of the following statements, or give a counterexample:

- b) Breadth-first search is a special case of uniform-cost search.
- c) Depth-first search is a special case of best-first tree search.
- d) Best-first search is optimal in the case where we have a perfect heuristic (i.e., $h(n) = h^*(n)$, the true cost to the closest goal state).
- e) Suppose there is a unique optimal solution. Then, A* search with a perfect heuristic will never expand nodes that are not in the path of the optimal solution.
- f) A* search is optimal in the case where negative edge weights are allowed.

Question 3: Search for Optimization [40]

Consider the following functions:

$$f_1(x,y) = \sin(2x) + \cos(y/2)$$

$$f_2(x,y) = ||x-2|| + ||0.5y+1|| - 4$$

We would like to maximize these functions within the range of $0 \leq x, y \leq 10$. For each part below and each setting, report the mean and standard deviation of the number of steps to convergence and of the final value f_1^* , f_2^* for each case.

We encourage you to try doing this problem yourself first, before turning to external sources of assistance. It will be easiest to write code to solve this problem. Use plots and/or tables to report your results in an organized manner.

- a) For each function, apply hill climbing, starting from 100 random points in the range. Repeat this procedure for each choice of step size in $[0.01, 0.05, 0.1, 0.2]$. A neighbour is a point where x and/or y has increased or decreased by the stepsize; i.e., there are up to 8 neighbours from any given point. What patterns do you see?

- b) Repeat using local beam search with beam width in $[2, 4, 8, 16]$, performing 100 runs of each. Were you able to improve performance over hill climbing for each function, as measured by the mean and standard deviation of the number of iterations and/or final objective function value?